

FINAL REPORT FOR FINAL PROJECT
ADVANCED CPU ARCHITECTURE AND HARDWARE

ACCELERATORS LAB

361.1.4693

Yoav Tsur 211903406

Elad Lavi 211626551

2026 Semester B

תוכן עניינים

3	מבוא למערכת הכוללת
5	חלק ראשון - הליבה RISC-V Core
9	תוספת המחלק לליבה כמאיץ חומרתי
13	ולידציה עבור פעולת המחלק
15	חלק שני - GPIO Peripherals
18	ולידציה עבור רכיבי הGPIO
20	חלק שלישי - פסיקות - Interrupts
26	ולידציה עבור רכיבי Interrupt
29	חלק רביעי - הטמעת המערכת בכרטיס FPGA DE10
31	חלק חמישי - ניתוח ביצועים

מבוא למערכת הכוללת

במעבדה זו נממש מערכת מורכבת המתבססת על ליבת מעבד הסינגל סייקל ממעבדה מספר 5 ונרחיב אותה למערכת שלמה המבצעת גם חלוקה ושארית, ובנוסף התמודדות עם בקשות פסיקה תוך כדי שגרה שוטפת, תקשורת שוטפת לGPIO וסנכרון בין שעונים בתדרים שונים (CDC).
להלן ליבת המעבד לאחר ההרחבה:

ליבה זו מרחיבה את ליבת המעבד סינגל סייקל לתמיכה בפעולת חילוק בין שני מספרים UNSIGNED הדורשת הוספת מאיץ חומרתי ייעודי לשם כך והוספת לוגיקה. בנוסף כפי שניתן לראות מן התמונה, מודול המחלק מוזן משעון שונה divclk ולשם כך נדרש סנכרון בין פעילות המעבד השוטפת המוזנת מן השעון mclk לבין שעון המחלק.

כעת נציג את המערכת הכוללת:

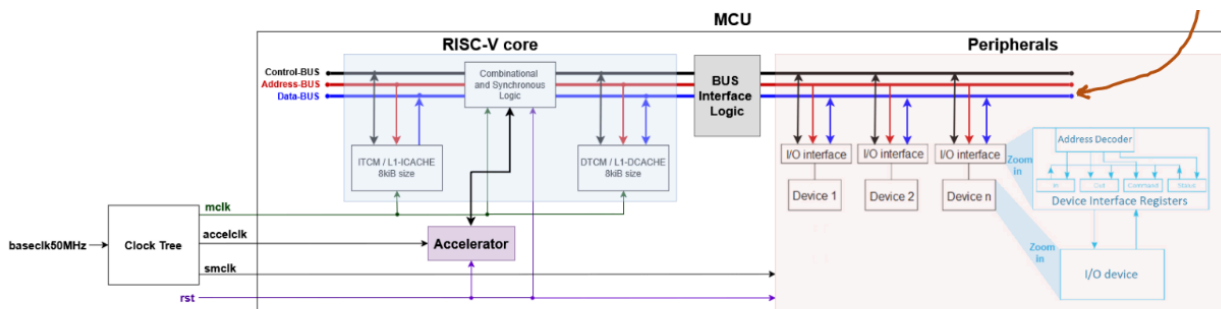


Figure 1: MCU System uArch

כפי שניתן לראות באיור, המערכת כולה מתוכננת כמיקרו-בקר שלם הכולל הפרדה פיזית בין זיכרון התוכנית לבין זיכרון הנתונים. הקישוריות בין הליבה לבין שאר חלקי המערכת מתבצעת דרך מערכת באסים הכוללת את באס הכתובות, באס הנתונים ובאס הבקרה. דרך קווים אלו, המעבד מנהל את התקשורת מול הזיכרון ומול הלוגיקה המקשרת לרכיבים הפריפריאליים.

אפיק הנתונים הפנימי של הליבה מקושר באמצעות באס נתונים דו-כיווני משותף ברוחב 32 ביט. בנוסף נעשה שימוש ב-Tri-state buffers, באמצעות רכיב BidirPin כאשר אנחנו מווסתים את היציאות של כתיבת נתוני המעבד וקריאת נתוני ה-DTCM אל הבאס, ובכך מונעים Bus Contention.

התקשורת מול רכיבי קלט/פלט (I/O interface) מתבצעת בשיטת Memory Mapped I/O, שבה הרכיבים החיצוניים ממפים כחלק ממרחב הכתובות של זיכרון הנתונים. מרחב כתובות זה מחולק לשני אזורים לוגיים עיקריים: החלק התחתון מוקצה לזיכרון הנתונים הפיזי, ואילו החלק העליון שמור לאוגרים ייעודיים (SFR - Special Function Registers) המשמשים את רכיבי הפריפריה השונים. הגישה לרכיבי ה-GPIO ולזיכרון עצמו נעשית על ידי מפענח כתובות חומרתי, אשר מנתח את הכתובות בת 14 הביטים שמייצר המעבד וקובע בזמן אמת איזה רכיב מופעל. חלוקת מרחב כתובות זה מוצג באיור הבא:

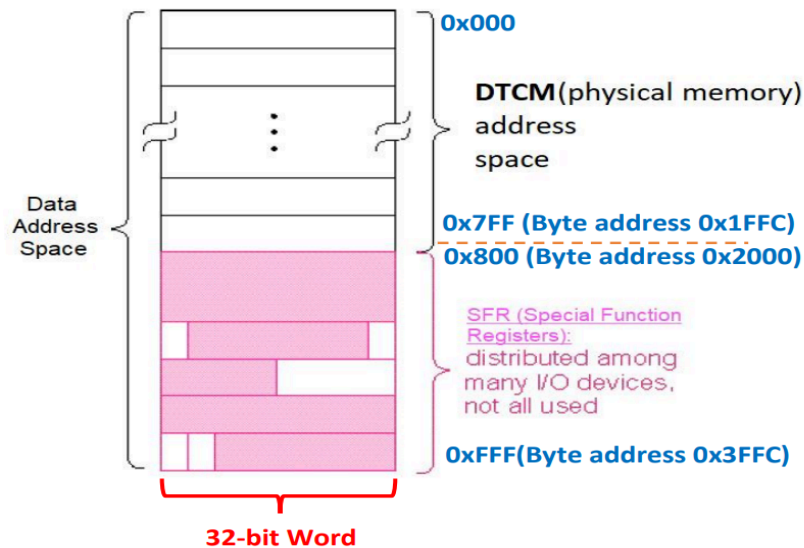


Figure 2: Data Address Space Map

בנוסף לממשק ה-I/O, המערכת מורחבת על ידי הוספת רכיבים פריפריאליים מתקדמים התומכים בפסיקות, ביניהם טיימר בסיסי בעל יכולות השוואה והפקת אותות PWM, ובקר פסיקות. מנגנון הפסיקות מאפשר למעבד להשהות את שגרת הביצוע השוטפת שלו על מנת להגיב לאירועים חיצוניים בזמן אמת. כאשר מתקבלת דרישת פסיקה מאחד ממקורות הפריפריה, בקר הפסיקות שולח אות בקשה (INTR) למעבד. בתגובה, המעבד משיב באות אישור (INTA) וקופץ לטפל באירוע בהתאם לרמת העדיפות המוגדרת עבורו בטבלת הפסיקות.

לשם ניהול זמנים והפעלה תקינה של כלל תתי-המערכות, מיושם במערכת עץ שעונים. עץ השעונים מקבל שעון בסיס של 50MHz ומפיק ממנו מספר תחומי תדר שונים הנדרשים למודולים השונים: שעון ה-mclk המזין את ליבת המעבד המרכזית, שעון ה-smclk עבור רכיבי הפריפריה, ושעון ה-accelclk המזין את המאיץ החומרת. חלוקה זו של שעונים בתדרים שונים מחייבת יישום מנגנוני סנכרון על מנת להבטיח העברת נתונים אמינה ולמנוע מצבי מטא-סטביליות.

חלק ראשון - הליבה RISC-V Core

ארכיטקטורת הליבה הכוללת (RV32IMscMCU):
ליבת המעבד ממומשת כמודל VHDL מבני בקובץ RV32IMscMCU.vhd.
הליבה משלבת ארכיטקטורת זיכרון מסוג הארוורד. הליבה פועלת כולה בארכיטקטורת מחזור-יחיד למעט הרחבת מאיץ החילוק הרב-מחזורי.
כל פקודה רגילה נשלפת, מפוענחת, מבוצעת, ותוצאותיה נכתבות חזרה לRegister File או לזיכרון ה-DTCM בתוך מחזור שעון אחד.
נשים לב כי קובץ הבקרה (control.vhd) כולל בתוכו גם את מכונת המצבים לטיפול בפסיקות והתאמת הבקרה והלוגיקה הנדרשת למערכת כולה, כמו כן בוצעו התאמות לשראר חלקי המעבד על מנת לתמוך בזרימה כוללת תקינה כנדרש.

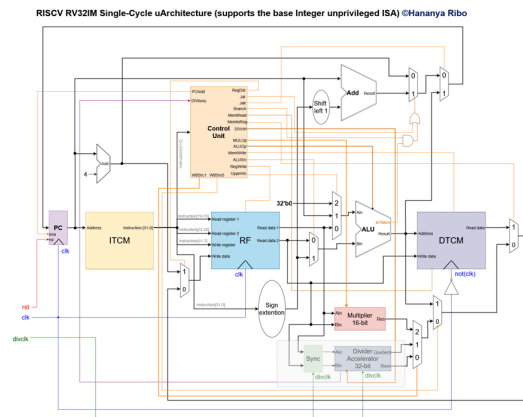


Figure 3: RISC-V RV32IM uArch

קבצי VHD בסיסיים המרכיבים את הליבה:

IFETCH.VHD .1

שלב זה אחראי על יצירת הכתובת הבאה של מונה התוכנית (PC) ושליפת הפקודה (32 ביט) מזיכרון ה-ITCM. בשלב זה ישנם מספר רכיבי חומרה הנמצאים בשימוש:
מונה התוכנית (PC): בגודל 32 ביט השומר את כתובת הפקודה הנוכחית. מתעדכן בעליית השעון, אלא אם כן דגל ההשהיה PChold פעיל.
Sequential Adder מחשב את הכתובת הבאה (PC+4) במקביל לביצוע הפקודה הנוכחית, ומכין את כתובת ברירת המחדל לפקודה העוקבת.
Target Address Multiplexers בוררים את מקור הכתובת הבאה של ה-PC בהתאם לאותות הבקרה משלב Execute ומיחידת הבקרה.
פסיקה (Interrupt Target): כאשר האות IntJump_ctrl_i פעיל (בזמן קפיצה לשגרת טיפול בפסיקה), ה-PC מדלג לכתובת הווקטור הנקראת מתוך טבלת הפסיקות (IVT).
באמצעות ה-ITCM מזהים את ה-Instruction שיש לבצע.

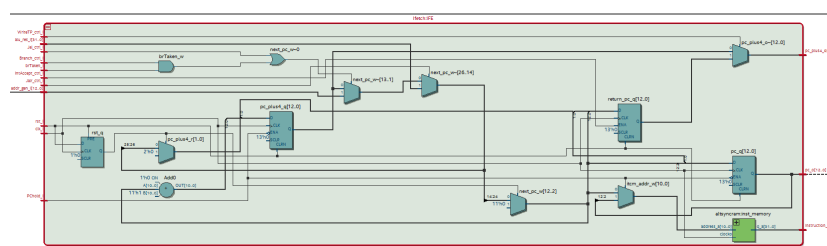


Figure 4: RV32IM Fetch Unit RTL

2. IDECODE.VHD

שלב זה אחראי על פירוק הפקודה שנשלפה והעברת השדות ליחידת הבקרה (שנמצאת בשימוש בשלב הדיקוד גם כן), קריאת האופרנדים מתוך ה-Register File והפקת ערך ה-Immediate בהתאם לפורמט הפקודה.

Register File: מכיל 32 אוגרים כלליים בגודל 32 ביט (x0-x31). הקריאה מתבצעת דרך שני פורטי קריאה בהתאם לשדות rs1 ו-rs2 של הפקודה. כתיבה לאוגר היעד rd מתבצעת בעליית השעון כאשר Reg Write_ctrl פעיל. האוגר x0 נשאר קבוע בערך 0.

Immediate / Sign Extension: המודול מחלץ את שדה ה-Immediate בהתאם לסוג הפקודה, I, S, B, U, J ומרחיב אותו ל-32 ביט תוך שמירה על הסימן.

Write-Back MUX: הנתון הנכתב ל-RF נבחר בין תוצאת ה-ALU, נתון שנקרא מה-DTCM או כתובת החזרה PC+4, בהתאם לאותות הבקרה.

בקרת פסיקות: הביט 0 באוגר x3 משמש כ-Global Interrupt Enable GIE. בעת כניסה לטיפול בפסיקה הביט מאופס, ובחזרה מהפסיקה הוא נקבע מחדש ל-1. בנוסף, בעת כניסה לפסיקה ניתן לכפות את x4 כאוגר היעד לצורך שמירת כתובת החזרה.

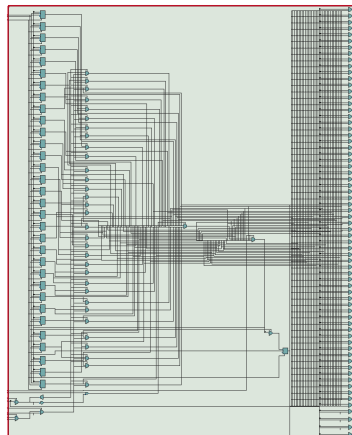


Figure 5: RV32IM Decode Unit RTL

3. EXECUTE.VHD

שלב זה אחראי על ביצוע הפעולות האריתמטיות, הלוגיות ופעולות ההזזה על האופרנדים. בנוסף, הוא מעריך תנאי קפיצה ומחשב את כתובות היעד.

Operand Source MUXs: בוררים את הכניסות ain_w ו-bin_w ליחידת ה-ALU.

ain_w בורר בין ערך האוגר read_data1 לבין כתובת ה-PC הנוכחית, ו-bin_w בורר בין read_data2 לבין sign_extend.

יחידת ALU: מבצעת את הפעולות בהתאם לאות ALUOp ctrl, כולל פעולות אריתמטיות, לוגיות, הזזה והשוואה.

M-extension Result MUXs: לצורך תמיכה בפקודות MUL, DIV ו-REM, האות WBSrc1 בוחר בין RESIDUE עבור REM, QUOTIENT עבור DIV, ותוצאת המכפל resmul_w עבור MUL. לאחר מכן, WBSrc0 בוחר בין תוצאת ה-M-extension לבין תוצאת ה-ALU הרגילה (alu_res_r), והתוצאה הנבחרת מועברת דרך alu_res_o להמשך ה-datapath.

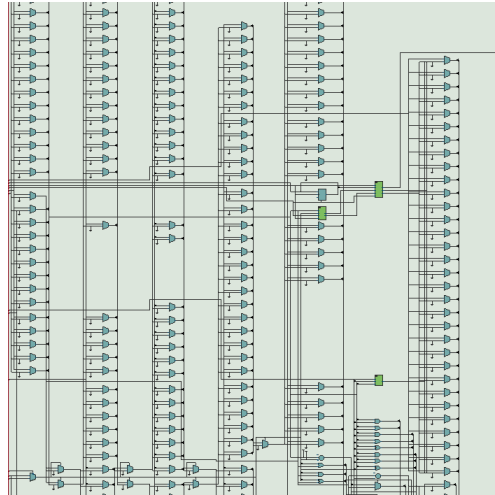


Figure 6: RV32IM Execute Unit RTL

DMEMORY.VHD .4

מודול זה מממש את זיכרון הנתונים DTCM, שבו מאוחסנים משתני הנתונים של התוכנית. שעון הזיכרון: שעון הכתיבה לזיכרון הוא השעון ההפוך של שעון המעבד ($wrclk_w \leq \text{not } clk_i$). לכן פעולת הכתיבה מתבצעת בחזית הנגדית לזו שבה מתעדכנים רכיבי המעבד, וכך ניתנת לכתובת ולנתוני הכתיבה אפשרות להתייצב לפני הכתיבה לזיכרון כיוון שאנחנו "מחלקים את המחזור" לשני חלקים. נתיב כתיבה: כאשר MemWrite ctrl פעיל, הערך $dtcm_data_wr$ נכתב לתא הזיכרון הנבחר על ידי $dtcm_addr$. נתיב קריאה: הערך שנמצא בכתובת $dtcm_addr$ מתקבל ישירות ביציאה $dtcm_data_rd_o$. בהמשך המערכת, אותות הבקרה קובעים מתי נתון זה משמש את המעבד או מועבר ל-DataBUS.

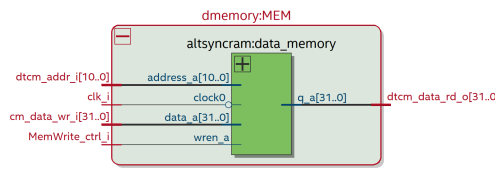


Figure 7: RV32IM Memory Unit RTL

multiplier_16bit.vhd.5

מודול זה מממש מכפל קומבינטורי בגודל 16×16 ביט, והוא משולב בשלב ה-EXECUTE לצורך ביצוע פעולת כפל. המכפל מקבל את 16 הביטים הנמוכים של שני האופרנדים ומפיק תוצאה בגודל 32 ביט. מכיוון שמדובר במימוש קומבינטורי, החישוב אינו דורש מכוונת מצבים או מספר מחזורי שעון. אלגוריתם הכפל: כל אחד מהאופרנדים מחולק לחלק עליון ותחתון של 8 ביט. לאחר מכן מתבצעים ארבעה כפליים מקבילים בגודל 8×8 ביט:

$$P0 = A_{low}B_{low} \quad P1 = A_{low}B_{high}, \quad P2 = A_{high}B_{low}, \quad P3 = A_{high}B_{high}$$

התוצאה הסופית מתקבלת מחיבור המכפלות החלקיות לאחר הזזה למקומן המתאים. האות mulOP קובע האם תוצאת המכפל תועבר למוצא rd, כאשר לא מתבצעת פעולת כפל, המוצא מאופס.

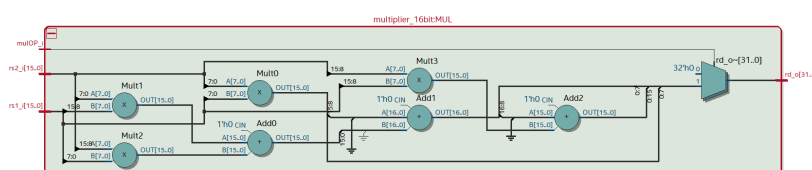


Figure 8: RV32IM Multiplier Unit RTL

6. CONTROL.VHD

מודול זה משמש כיחידת הבקרה המרכזית של המעבד. הוא מפענח את הפקודה הנוכחית ומייצר את אותות הבקרה הדרושים לשאר יחידות ה-Datapath, בהתאם לסוג הפקודה ולפעולה הנדרשת. Instruction Decode: הפקודה מפוענחת לפי שדות ה-Opcode וה-Function לצורך זיהוי פעולות אריתמטיות ולוגיות, Branch, Load/Store, Jump וכן פקודות MUL, DIV ו-REM. Control Signals: בהתאם לפקודה, המודול מייצר את אותות הבקרה כגון, RegWrite, MemRead, MemWrite, ALUSrc, MemtoReg, Branch, Jal, Jalr האותות הנדרשים למוקסים ולכתיבה/קריאה. בקרת M-Extension: עבור MUL, DIV ו-REM נוצרים האותות WBSrc0, WBSrc1, MULop, DIVctrl, המפעילים את יחידת הכפל או החילוק ובוחרים את התוצאה המתאימה להמשך ה-Datapath. השהיית המעבד בזמן חילוק: פעולת החילוק היא פעולה רב-מחזורית, ולכן PChold_ctrl משמש לעצירת התקדמות ה-PC בזמן שהמחלק פעיל. בנוסף, div_started_q עוקב אחר תחילת פעולת החילוק ומאפשר לבצע Stall כבר במחזור הראשון, לפני ש-DIVbusy עולה, וכן מונע הפעלה חוזרת של המחלק בזמן שהפקודה מוחזקת. המודול כולל גם את לוגיקת הבקרה הנדרשת לטיפול בפסיקות ולפרוטוקול INTR/INTA, אשר תפורט בנפרד בפרק העוסק במערכת הפסיקות.

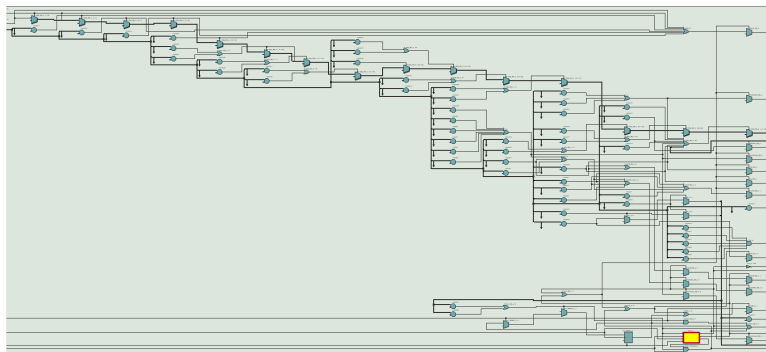


Figure 9: RV32IM Control Unit RTL

ובנוסף מצורף RTL כללי של הליבה המכיל את כל הרכיבים אשר פירטנו עליהם. כפי שציינו ליבה זאת מתחברת לממשק ה-TOP באמצעות TriState לצורך מניעת התנגשות על ה-BUS.

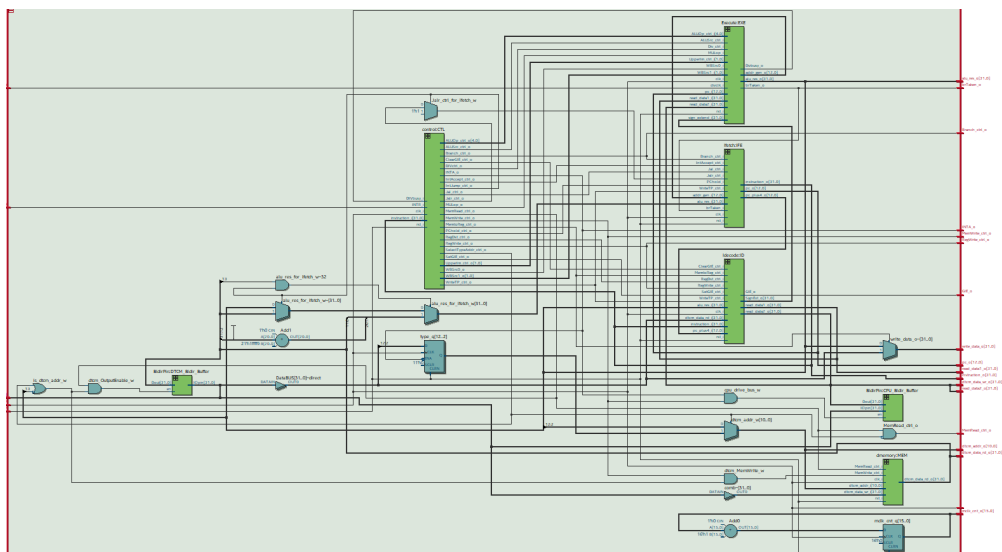


Figure 10: RV32IM Unit RTL

תוספת המחלק לליבה כמאיץ חומרתי

מימוש המחלק 32bit.vhd:divider

בחלק הראשון של הפרויקט נדרשנו לממש מאיץ חומרה לביצוע פעולות חילוק ושארית (DIV ו-REM) עבור מספרים Unsigned, כחלק מהרחבת ליבת ה-RV32IM. המחלק ממומש כ-Multi Cycle Division Accelerator המחובר לליבת המעבד ופועל בתחום שעון נפרד משעון המערכת. עובר אופרנדים בגודל 32 ביט ($N=32$), פעולת החילוק מתבצעת באופן איטרטיבי לאורך 32 מחזורי DIVCLK.

קלטי המחלק:

Dividend – המספר המחולק.

Divisor – המספר המחלק.

DIVCLK – שעון המאיץ.

Reset – איפוס המודול.

DIVENA – אות המתחיל פעולת חילוק.

מוצאי המחלק:

Quotient – מנת החילוק.

Residue – שארית החילוק.

DIVBUSY – אות המציין שהמחלק נמצא במהלך ביצוע פעולה ומשמש את יחידת הבקרה לצורך עיכוב המעבד עד לסיום החישוב.

אלגוריתם המחלק:

המחלק מממש אלגוריתם חילוק בינארי איטרטיבי עבור מספרים Unsigned, כאשר בפרויקט זה $N=32$. כאשר אות הבקרה DIVENA עולה בזמן שהמחלק פנוי, נטענים אופרנדי ה-Dividend וה-Divisor לרגיסטרים הפנימיים, רגיסטר המנה מתאפס, המונה מתחיל מ-0 ואות ה-DIVBUSY עולה ל-1 כדי לאותת שהיחידה עסוקה. המימוש החומרתי עושה שימוש ברגיסטר הזה בגודל $2N$ ביטים, כאשר מחציתו העליונה מחזיקה את השארית החלקית ומחציתו התחתונה מחזיקה את ביטי ה-Dividend שטרם עובדו. בכל מחזור של שעון ה-DIVCLK מתבצעת איטרציה אחת שבה השארית החלקית מוזזת שמאלה ומצורף אליה הביט הבא של ה-Dividend. הערך המתקבל מושווה ל-Divisor: אם הוא גדול או שווה לו, מתבצע חיסור (התוצאה אינה שלילית) ונוסף הביט 1 למנה, ואחרת לא מתבצע חיסור ונוסף הביט 0 למנה. התהליך חוזר על עצמו במשך 32 איטרציות, ולאחר מכן DIVBUSY יורד ל-0 והתוצאות הסופיות זמינות במוצאי ה-QUOTIENT (מנת החילוק) וה-RESIDUE (שארית החילוק). לאורך התהליך, האות DIVBUSY משמש את יחידת הבקרה של המעבד כדי להשהות את התקדמותו כל עוד פעולת החילוק הרב-מחזורית טרם הסתיימה.

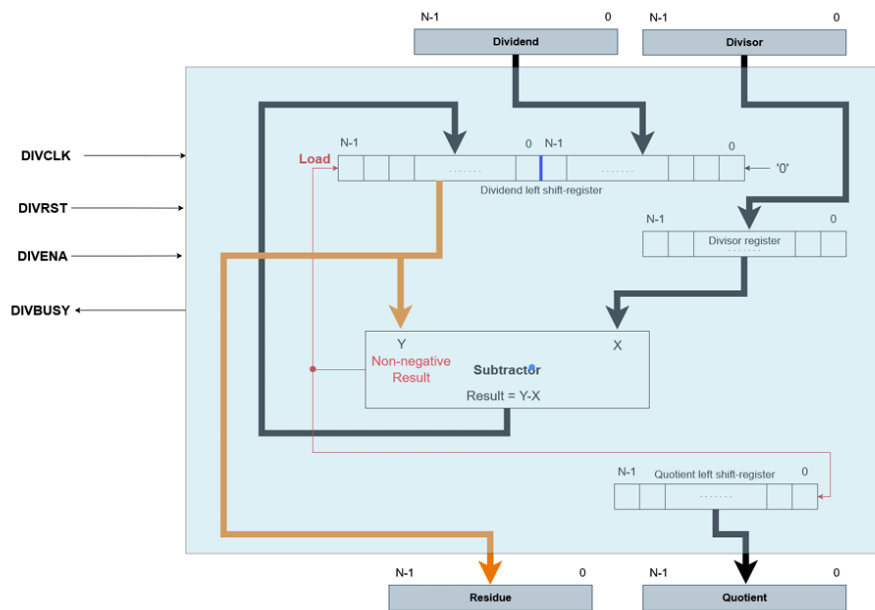


Figure 11: Multicycle-based architecture of Unsigned Divisor accelerator

מצורף צילום של הRTL עבור מודול ה-Divider32bit:

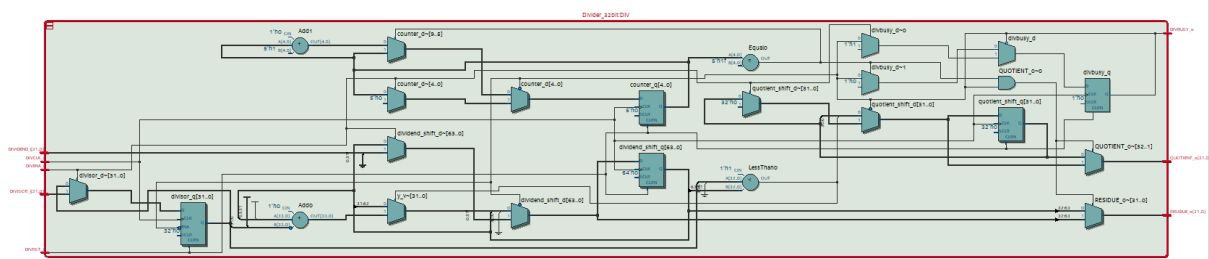


Figure 12: RV32IM Divider Unit RTL

מסנכרן :sync.vhd

המחלק פועל באמצעות DIVCLK, בתדר שונה משעון המערכת של המעבד. לעומת זאת, האופרנדים המועברים למחלק (Divisor ו- Dividend) מגיעים מליבת המעבד ולכן שייכים ל-Clock Domain של שעון המערכת.

כתוצאה מכך נוצר Clock Domain Crossing CDC. ללא סנכרון, שינוי באחד האופרנדים עלול להתרחש בסמוך לחזית העולה של DIVCLK, ובכך להפר את זמני ה-setup/hold של רגיסטר במחלק ולגרום למצב של Metastability. לכן לפני העברת האופרנדים למחלק הם מועברים דרך מודול Synchronizer.

אופן פעולת המסנכרן

במימוש שלנו כל אחד משני האופרנדים עובר דרך שתי דרגות של רגיסטרים, הפועלות לפי DIVCLK:

$Ain_i \rightarrow Ain_q1 \rightarrow Ain_q2 \rightarrow Ain_o$

$Bin_i \rightarrow Bin_q1 \rightarrow Bin_q2 \rightarrow Bin_o$

הדרגה הראשונה קולטת את האות המגיע מתחום השעון של המעבד ועלולה להיכנס לזמן קצר למצב metastable, ואילו הדרגה השנייה דוגמת את הערך לאחר שניתן לו זמן להתייצב. כך קטן משמעותית הסיכון להעברת ערך לא יציב אל המחלק.

המסנכרן מטפל בבעיה של מעבר בין שעונים:

CPU clock domain $\rightarrow$ Synchronizer $\rightarrow$ DIVCLK domain

לעומת זאת, DIVBUSY מטפל בכך שהחילוק לוקח 32 מחזורי DIVCLK. בזמן $DIVBUSY=1$ יחידת הבקרה משהה את התקדמות המעבד, כדי שה-CPU לא יעבור לפקודה הבאה לפני שתוצאת DIV/REM מוכנה.

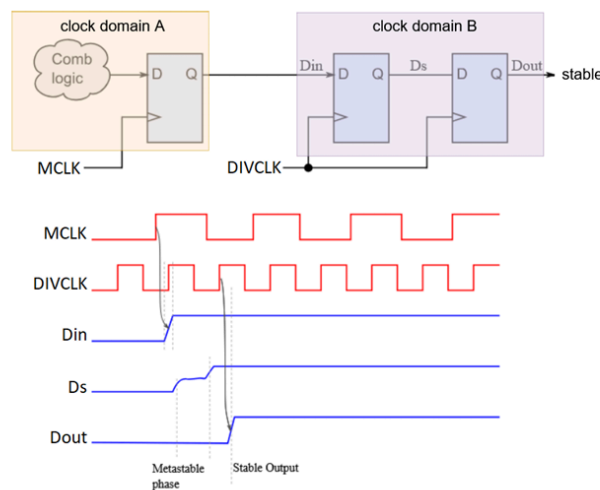


Figure 13: CDC (clock domain crossing) synchronizer

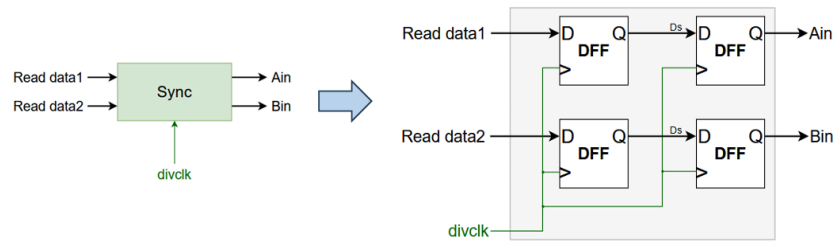


Figure 14: Synchronizer architecture

מצורף צילום של הRTL עבור מודול ה-Sync:

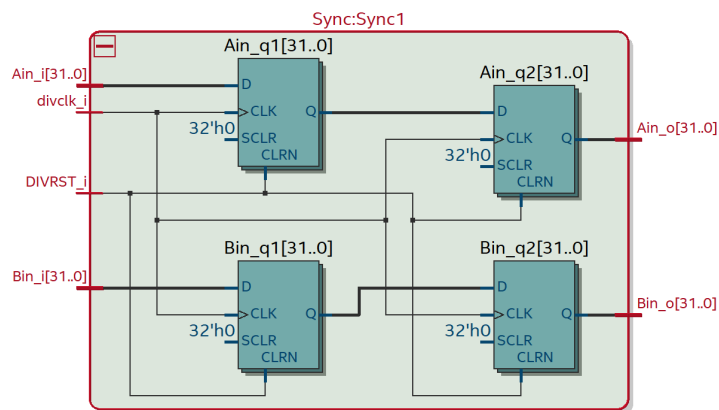


Figure 15: RV32IM synchronizer Unit RTL

ולדיציה עבור פעולת המחלק

על מנת לבדוק שהמחלק פועל כמו שצריך נדרשנו לבדוק התאמה ופעילות נכונה של המעבד באמצעות סטט שניתן לנו מראש.

הסטט מכיל 131 הוראות מתוכן 11 הוראות הן עבור טעינת הנתונים. שאר ההוראות בנויות מלולאה באורך 8 כך שכל סיבוב של הלולאה מונה 15 פקודות - 13 פקודות רגילות, פקודת חילוק ופקודת שארית.

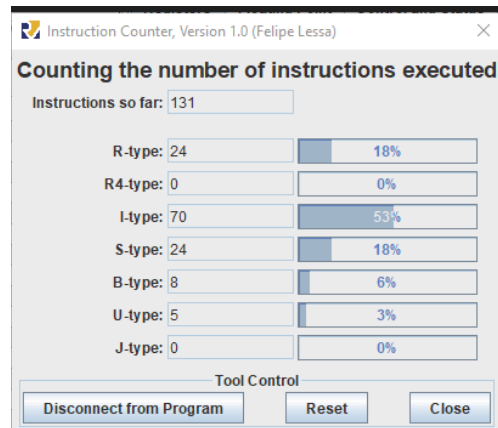


Figure 16: Divisor test instruction distribution

להלן תוצאות ה-Waveform במודל סימ:

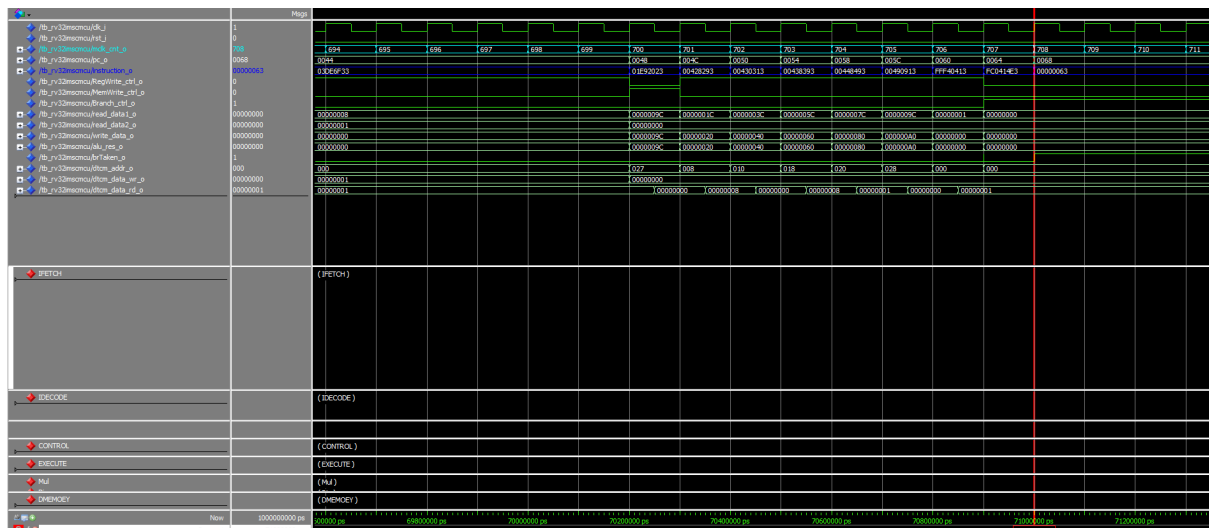


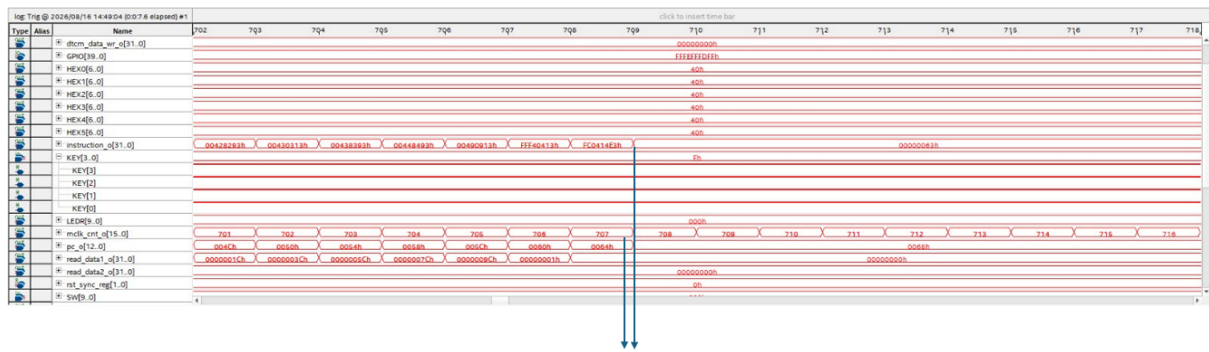
Figure 17: ModelSim Verification of the Division Accelerator

זמן ריצה זה תואם במדויק את המצופה ממבנה הקוד: התוכנית נפתחת ב-11 פקודות של טעינת נתונים, אשר כל אחת מהן מתבצעת במחזור שעון יחיד (Single Cycle). לאחר מכן, מתבצעת לולאה החוזרת על עצמה במשך 8 איטרציות. בכל איטרציה של הלולאה ישנן 15 פקודות: 13 פקודות רגילות (הדורשות מחזור שעון אחד כל אחת), פקודת חילוק ופקודת שארית. מכיוון שפעולות החילוק והשארית מבוצעות על ידי המאיץ הרב-מחזורי, כל אחת מהן עוצרת את המעבד ואורכת 37 מחזורי שעון. מכאן נובע כי כל איטרציה בלולאה דורשת 87 מחזורי שעון (37 + 37 + 13). סך מחזורי השעון של התוכנית כולה מורכב מ-11 מחזורי הטעינה ההתחלתיים בתוספת 8 איטרציות של 87 מחזורים מה שמוביל בדיוק ל-707 מחזורי שעון. התאמה מלאה זו מאשרת כי מנגנון הסנכרון וההשהיה של המעבד מול המאיץ פועל כנדרש.

$$IPC = \text{Instruction/Cycles} \rightarrow 131/707 = 0.185$$

הדבר הגיוני כי אמנם למדנו וראינו במעבדה 5 כי ב-Single Cycle צריך לצאת IPC של 1, אך מכיוון והוספנו מאיץ חלוקה רב מחזורי הגיוני כי ה-IPC יצא קטן מ-1 מאחר ואנו ממתינים זמן רב.

להלן תוצאות ה־Signal Tap :



ניתן לראות כי הסיום מגיע במחזור 707

Figure 18: STP Validation of the Division Accelerator

ניתן לראות כי גם בבדיקת זמן אמת על FPGA באמצעות Signal TAP קיבלנו תוצאות זהות. נציין כמובן כי הן במודל סים והן ב־QUARTUS תחת ISMCE נבדקו הזכרונות DTCM לאחר הרצת התוכנית בוצע השוואה ב־Golden Moudel מול ה־RARS לצורך ביצוע נכון ומדויק של תוצאות הטסט.

חלק שני - הGPIO Peripherals

בעוד שה-CPU אחראי על שליפה, פענוח וביצוע פקודות, ה-MCU משלב את ליבת המעבד יחד עם זיכרונות והתקנים היקפיים המאפשרים תקשורת עם רכיבי הקלט והפלט של המערכת.

במימוש שלנו, הקובץ `FPGA_Top_SingleCycle.vhd` הרמה העליונה של המערכת ומחבר את ליבת RV32IMscMCU לרכיבים הפיזיים בלוח ה-FPGA, כגון המתגים, הלחצנים, הלדים ותצוגות Seven Segment על גבי ששת צגי ה-HEX בכרטיס.

בפרויקט זה נשתמש ב-`Key0` בתור ריסט המערכת, `Key1-3` כבקשות פסיקה ו-`SW0-9` כקלטים. מוצאי המערכת יהיו ב-`LED 0-9` ו-`HEX5-0`.

בנוסף יש `Expansion Header GPIO of 2x20 pins`

הנחיות מימוש ממשק ה-GPIO:

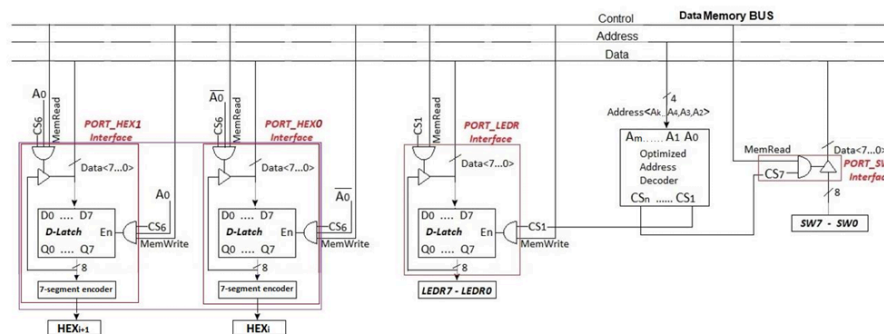


Figure 19: Basic GPIO peripheral connection using Memory Mapped I/O

בנוסף, ישנו צורך בשימוש של Address Decoder מכיוון שרכיבי ה-GPIO ממומשים בשיטת Memory-Mapped I/O. כלומר, מבחינת ה-CPU, גישה ל-LEDs, ל-HEX או ל-Switches נראית כמו גישה שגרתית לכתובת בזיכרון. לכן נדרש רכיב חומרתי שמקבל את הכתובת שהמעבד מוציא אל הבאס, מנתח אותה, וקובע לאיזה התקן בדיוק הגישה מיועדת. בפועל, המפענח בוחן את ביטי הכתובת ומפיק אות בחירת רכיב (Chip Select - CS) ייעודי עבור ההתקן המבוקש. אות זה, בשילוב עם אותות הבקרה לקריאה (MemRead) או לכתיבה (MemWrite), מבטיח שרק הרכיב הרלוונטי יגיב לפקודת המעבד בעוד שאר הרכיבים בנתק. להלן מיפוי הכתובות לפי רכיבים:

Memory-mapped GPIO <u>without</u> interrupt capability				
I/O device name	I/O mapping address	Note	Address Resolution	Direction
PORT_LEDR	0x2000	LEDR7- LEDR0	Byte address	GPO
PORT_HEX0	0x2004		Byte address	GPO
PORT_HEX1	0x2005		Byte address	GPO
PORT_HEX2	0x2008		Byte address	GPO
PORT_HEX3	0x2009		Byte address	GPO
PORT_HEX4	0x200C		Byte address	GPO
PORT_HEX5	0x200D		Byte address	GPO
PORT_SW	0x2010	SW7- SW0	Byte address	GPI

Table 20: Memory-Mapped Addresses for GPIO Peripherals

קבצי VHD המרכיבים את GPIO Peripherals:

GPIO_peripherals.vhd.1

מודול זה משמש כחוזר בין ליבת המעבד להתקני הקלט והפלט של הלוח באמצעות Memory-Mapped I/O. הוא אחראי על פענוח הכתובות, שמירת ערכי הפלט והעברת ערכי הקלט אל ה-DataBUS. Address Decoder: המודול בודק האם הכתובת נמצאת בתחום ה-0x201F-0x2000 GPIO ומייצר אות Chip Select עבור ההתקן המתאים. הכתובת 0x2000 ממופה ל-LEDR, הכתובות 0x2004-0x200D ממופות לתצוגות HEX0-HEX5, ו-0x2010 ממופה למתגים SW. עבור זוגות ה-HEX נעשה שימוש גם בביט הכתובת הנמוך (0) addr_i כדי לבחור את התצוגה הספציפית. פעולת כתיבה: כאשר mem_write פעיל, שמות הביטים הנמוכים של DataBUS נשמרים ברגיסטר הפלט שנבחר על ידי ה-Address Decoder. כך ערכי ה-LEDs וה-HEX נשמרים גם לאחר סיום פעולת הכתיבה. פעולת קריאה: כאשר המעבד קורא מכתובת המתגים, ערך SW_pins_i מורחב ל-32 ביט ומועבר ל-DataBUS. לאחר סיום הקריאה המודול מפסיק לנהוג על ה-bus כדי למנוע התנגשות עם רכיבים אחרים.

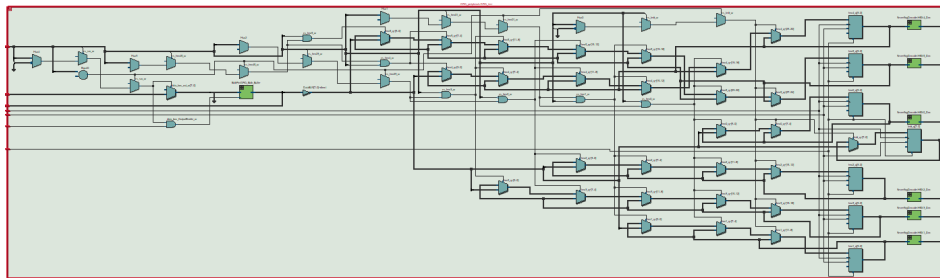


Figure 21: GPIO Unit RTL

BidirPin.vhd.2

מודול זה מממש חוץ דו-כיווני מסוג Tri-State עבור ה-DataBUS המשותף. מטרתו לאפשר למספר רכיבים במערכת להשתמש באותו bus מבלי ליצור התנגשות בין מקורות נתונים שונים. נתיב קריאה: האות Din מחובר ישירות ל-IOpin, ולכן המודול יכול לקרוא באופן רציף את הערך הקיים על ה-bus. נתיב כתיבה: כאשר 'en'=1, הערך Dout מועבר אל IOpin והמודול נוהג על ה-bus. כאשר 'en'=0, המוצא עובר למצב High-Impedance, וכך המודול מתנתק מה-bus ומאפשר לרכיב אחר לנהוג עליו. באמצעות מנגנון זה נמנעת Bus Contention, כלומר מצב שבו שני רכיבים מנסים לכתוב בו-זמנית ערכים שונים על אותו קו נתונים. נציין בנוסף כי יש שימוש במודל זה גם בתוך הליבה וגם ב-Interrupt Controller, כלומר כל רכיב אשר מחובר ל-Top משתמש במודל זה כדי למנוע התנגשות על ה-BUS ויצירת CDB.

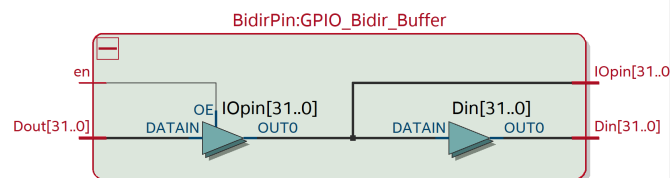


Figure 22: BidirPin Unit RTL

SevenSegDecoder.vhd .3

מודול זה משמש כמפענח קומבינטורי לתצוגת 7-Segment, הממיר ערך הקסדצימלי בגודל 4 ביט (F-0) לתבנית ההדלקה המתאימה של שבעת הסגמנטים. התצוגה פועלת בשיטת Active-Low, כלומר ערך '0' מדליק סגמנט וערך '1' מכבה אותו. בהתאם לערך הקלט, המודול בוחר באמצעות CASE את תבנית ה-seg המתאימה להצגת הספרה או האות ההקסדצימלית.

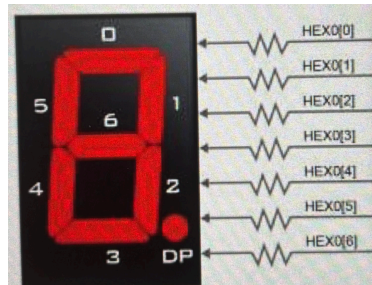


Figure 23: HEX mapping from Binary vector to Hex digit

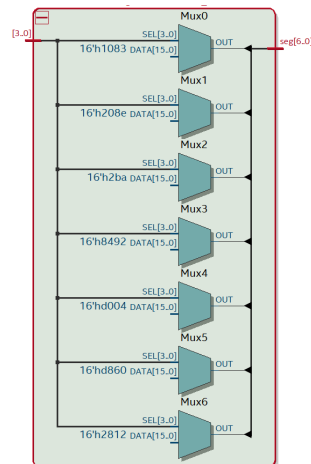


Figure 24: Seven Segment Display RTL

ולדיציה עבור רכיבי ה-GPIO

על מנת לבדוק שרכיבי ה-GPIO פועלים ומגיבים בהתאם לתוכנית ביצענו בדיקות שלוש הטסטים שקיבלנו. **טסט 0:**

טסט זה בודק את פעולת התקני הפלט של ה-GPIO. התוכנית מבצעת ספירה עולה ב-1 בצורה מחזורית, ומציגה את ערך המונה בנפרד על כל התקני הפלט: LEDR ו-HEX0-HEX5. כלומר כולם מציגים את אותה התוצאה בו זמנית.

כפתור KEY0 משמש לביצוע Reset למערכת והוא גם משמש כטריגר וציר האפס בולדיציה.



Figure 25: STP Validation of test0 GPIO

טסט 1:

טסט זה בודק את פעולת ה-GPIO גם כקלט וגם כפלט. לפי מצב המתגים SW, התוכנית משנה את כיוון הספירה ומציגה את הערך על LEDR ועל HEX0-HEX5. הטרירגרטסט זה היה שינוי במצב SW כאשר SW=0x01 מתבצעת ספירה עולה ב-1, כאשר SW=0x02 מתבצעת ספירה יורדת ב-1, ובכל ערך אחר הספירה נעצרת והערך האחרון נשמר בתצוגות. KEY0 משמש ל-Reset של המערכת.

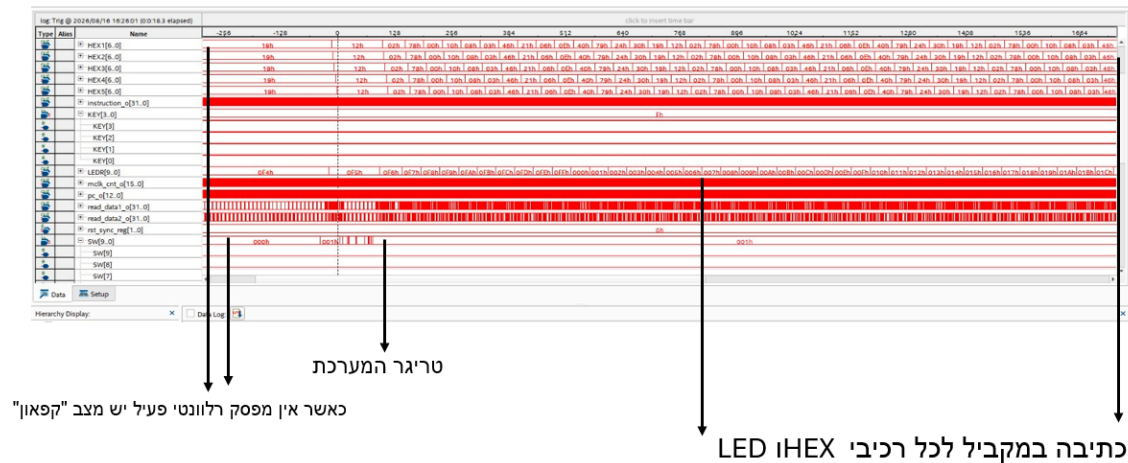
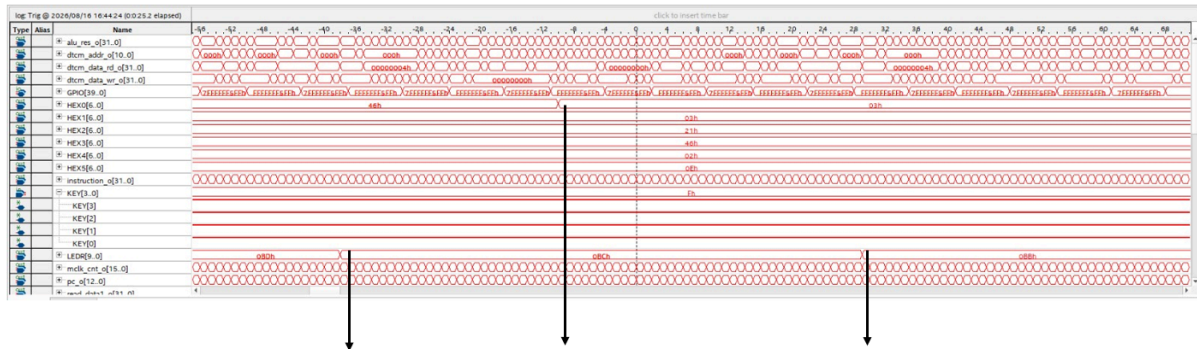


Figure 26: STP Validation of test1 GPIO

טסט 2:

טסט זה בודק את פעולת הקלט והפלט של ה-GPIO באמצעות המתגים והתצוגות. כאשר SW=0x01 מתבצעת ספירה עולה ב-1, וכאשר SW=0x02 מתבצעת ספירה יורדת ב-1. ערך המונה מוצג כמספר בן שש ספרות על מערך HEX5–HEX0 ובמקביל גם על LEDR 7:0. בכל ערך אחר של המתגים הספירה נעצרת והערך האחרון נשמר. KEY0 משמש ל-Reset של המערכת.



המספר מוצג כמספר בן 6 ספרות לכן לא כולם משתנים ביחד

Figure 27: STP Validation of test2 GPIO

חלק שלישי - פסיקות - Interrupts

מערכת הפסיקות מאפשרת להתקנים היקפיים לעצור באופן זמני את רצף הביצוע הרגיל של המעבד ולגרום למעבד ל-Interrupt Service Routine המתאים. במימוש שלנו קיימים ארבעה מקורות פסיקה: ה-Basic Timer ושלושת הלחצנים KEY1–KEY3. כל מקור מחובר ל-Interrupt Controller, האחראי על שמירת בקשות פסיקה ממתנות, אפשרות מקורות הפסיקה, קביעת סדרי עדיפויות ושליחת אות INTR למעבד.

Memory-mapped Peripherals <u>with</u> interrupt capability			
I/O device name	I/O mapping address	Address Resolution	Device
PORT_PB	0x2014	Byte address	KEY [3-1]
UTCL	0x2018	Byte address	USART
RXBF	0x2019	Byte address	USART
TXBF	0x201A	Byte address	USART
BTCTL1	0x201C	Byte address	Basic Timer
BTCTL2	0x201D	Byte address	Basic Timer
BTCMPR0	0x2020	Word address	Basic Timer
BTCMPR1	0x2024	Word address	Basic Timer
BTCAPR	0x2028	Word address	Basic Timer
IE	0x202C	Byte address	Interrupt Controller
IFG	0x202D	Byte address	Interrupt Controller
TYPE	0x202E	Byte address	Interrupt Controller

Figure 28: Memory-Mapped I/O addresses of interrupt-capable peripherals

כלל רכיבי הפסיקה וה-Basic Timer ממופים למרחב ה-Memory-Mapped I/O של המעבד. הטבלה מציגה את כתובות הרגיסטרים וההתקנים הרלוונטיים, באמצעות מיפוי זה המעבד יכול לקרוא ולכתוב לרכיבי הבקרה באמצעות פקודות Load/Store רגילות. שורות ה-USART המופיעות בטבלה אינן בשימוש במימוש שלנו, מאחר שבחרנו שלא לממש את בונוס ה-UART.

Interrupt Vector Table – IVT

ה-IVT הינה טבלה בזיכרון המכילה עבור כל מקור פסיקה את כתובת פונקציית השירות המתאימה (ISR). לכל מקור פסיקה מוגדר ערך TYPE קבוע, המשמש ככתובת הכניסה שלו בטבלת הווקטורים. לפי הגדרת הפרויקט, למקורות שבהם אנו משתמשים קיימים הערכים: 0x10 עבור ה-Basic Timer, 0x14 עבור KEY1, 0x18 עבור KEY2 ו-0x1C עבור KEY3.

Interrupts Vector Table				
Interrupt Source	Interrupt Flag	TYPE content	System Interrupt	Priority
RESET	-	00h	NMI	0, Highest
UART status error	RXIFG	04h	maskable	1
UART RX	RXIFG	08h	maskable	2
UART TX	TXIFG	0Ch	maskable	3
Basic Timer	BTIFG	10h	maskable	4
Pushbutton 1	KEY1	14h	maskable	5
Pushbutton 2	KEY2	18h	maskable	6
Pushbutton 3	KEY3	1Ch	maskable	7, Lowest

Figure 29: Interrupts Vector Table

חשוב להדגיש כי ה-TYPE אינו כתובת ה-ISR עצמה. ה-TYPE מצביע לכניסה המתאימה ב-IVT, ובאותה כניסה שמורה כתובת ה-ISR:

TYPE = 0x10 → Mem[0x10] → Timer ISR address

TYPE = 0x14 → Mem[0x14] → KEY1 ISR address

TYPE = 0x18 → Mem[0x18] → KEY2 ISR address

TYPE = 0x1C → Mem[0x1C] → KEY3 ISR address

בעת קבלת פסיקה, ה-Interrupt Controller מעביר את ערך ה-TYPE למעבד דרך ה-DataBUS. הערך נשמר ברגיסטר פנימי type_q, ובמחזור הטיפול הבא הוא משמש ככתובת גישה ל-DTCM במקום הכתובת הרגילה שמגיעה מה-ALU. הערך שנקרא הוא כתובת ה-ISR, והוא משמש לביצוע הקפיצה של פונקציית השירות.

קבצי VHD המרכיבים את חלק ה-Interrupts:

1. InterruptController.vhd

מודול זה מממש את בקר הפסיקות (INTC) של המערכת. מטרתו לנהל באופן ריכוזי את בקשות הפסיקה החומרתיות, לסנן אותן, לתעדף אותן ולתקשר מול הליבה לייצור פסיקה מאוחדת (INTR). המודול כולל בתוכו מנגנון Edge Detection: המודול קולט דרישות חומרתיות משלושת לחצני המשתמש (KEY1, KEY2, KEY3) ומדגל הפסיקה של הטיימר הבסיסי (BTIFG). הלחצנים מזוהים ב-Falling edge, ואילו פסיקת הטיימר מזוהה ב-Rising edge. ניהול רגיסטרים וסיון חומרת: אוגר ה-IE ממופה לכתובת 0x202C ואוגר ה-IFG לכתובת 0x202D. בהתאם להגדרות הארכיטקטורה המודול מבצע חסימה חומרתית בכתיבה כך שרק ביטים 5:2 ניתנים לעדכון מה-bus, בעוד שביטים 7:6 ו-1:0 מקובעים ל-'0' (מיועדים ל-UART). בנוסף, מיושם סינון המבטיח כי בקריאת האוגר יתקיים כי דגלי פסיקה ממתנים שאינם מאופשרים יוחזרו תמיד כ-'0'.

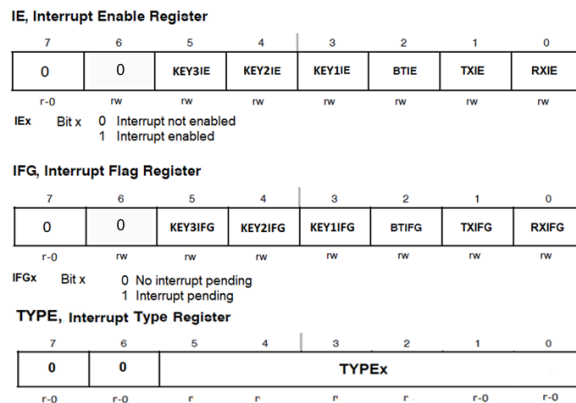


Figure 30: Interrupt Registers Structure

בהתאם לדרישות הפרויקט קובץ זה אחראי גם על יישום Priority Encoder: תהליך קומבינטורי הבוחן את הדגלים הפעילים לאחר הסינון וקובע את הפסיקה בעלת העדיפות הגבוהה ביותר להגשה (ניתן לראות את פירוט עדיפויות המלא ב-Table 29). מבין הפסיקות שאינן NMI, פסיקת טיימר מוגדרת כעדיפות העליונה, ואחריה הלחצנים KEY1 עד KEY3 לפי סדר עולה. במידה וקיימת פסיקה פעילה, ובכפוף לאות האפשרות הגלובלי המגיע מהליבה (GIE) שמגיע מהמעבד מועלה אות ה-INTR.

Handling interrupts from several sources:

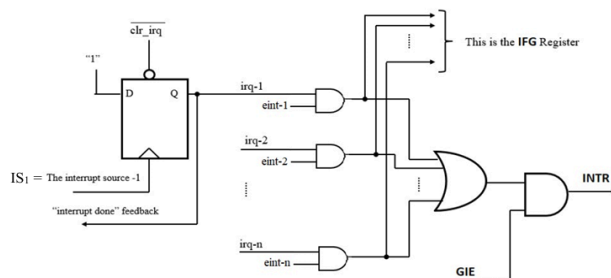


Figure 31: Hardware Logic for Handling Multiple Interrupt Sources

מענה לפסיקה (INTA) ושליטה על ערוץ הנתונים: פרוטוקול אישור הפסיקה נשלט באמצעות האות INTA. כאשר INTA יורד ל-'0', המודול נועל את וקטור סוג הפסיקה הפעיל אל אוגר `type_q`. במקביל, המודול משתלט על ה-DataBUS הדו-כיווני ודוחף את הוקטור למעבד, תוך עקיפת פעולת קריאת זיכרון סטנדרטית. איפוס אוטומטי: על מנת למנוע כניסה חוזרת לשגרת ה-ISR, המודול דוגם את קו ה-INTA לזיהוי עליית מתח. ברגע סיום המעבר של INTA חזרה ל-'1', מתבצע איפוס חומרתי סינכרוני לדגל הפסיקה הספציפי.

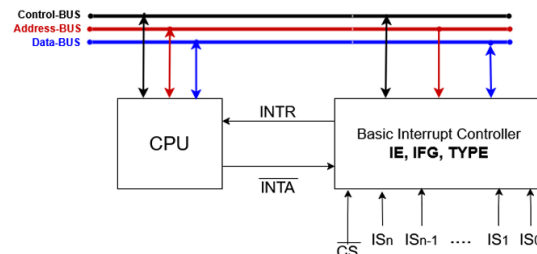


Figure 32: CPU and INTC Handshake and Bus Connections

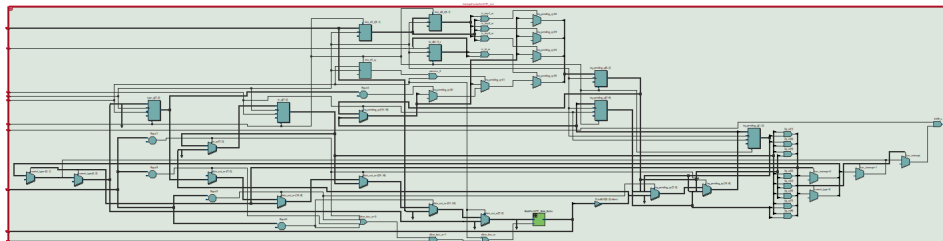


Figure 33: RTL Interrupt Controller Unit

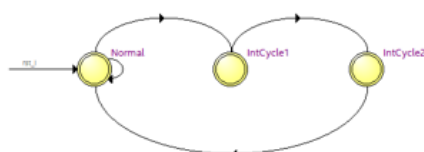
control.vhd.2

ה-FSM שנמצא ביחידת הבקרה מנהל את רצף הכניסה לפסיקה והמעבר ל-ISR. הוא כולל שלושה מצבים: `IntCycle1`, `Normal` ו-`IntCycle2`.

Normal: מצב העבודה הרגיל של המעבד. פסיקה מתקבלת כאשר `INTR = 1` והמחלק אינו עסוק (`DIVbusy = 0`). במקרה זה `accept interrupt` עולה, ה-PC נעצר, כתובת החזרה נשמרת וה-FSM עובר ל-`IntCycle1`.

IntCycle1: במצב זה INTA יורד ל-0 ובכך מאותת ל-Interrupt Controller שהפסיקה התקבלה. במקביל `ClearGIE` מאפס את `GIE` כדי למנוע קבלת פסיקות נוספות, וה-Interrupt Controller מציב את ערך ה-`TYPE` על ה-DataBUS, ממנו הוא נשמר ב-`type_q`.

IntCycle2: ערך ה-`TYPE` משמש לגישה ל-Interrupt Vector Table. האות `Select Type Addr` בוחר את `TYPE` ככתובת ל-DTCM, והערך הנקרא מ-`Mem[TYPE]` הוא כתובת ה-ISR. במקביל `WriteTP` שומר את כתובת החזרה ב-`4x`, ו-`IntJump` מבצע את הקפיצה ל-ISR. לאחר מכן ה-FSM חוזר ל-`Normal`. בסיום ה-ISR מתבצעת הפקודה `jalr x0, 0(tp)`. ה-CONTROL מזהה אותה כ-`RETI`, מפעיל `SetGIE` ומאפשר מחדש קבלת פסיקות, תוך חזרה לכתובת שנשמרה ב-`tp` לטובת המשך ריצת התוכנית המקורית במעבד.



Source State	Destination State	Condition
1 IntCycle1	IntCycle2	
2 IntCycle2	Normal	
3 Normal	Normal	$(\overline{DIVbusy_i}) \cdot (\overline{INTR_i}) + (DIVbusy_i)$
4 Normal	IntCycle1	$(DIVbusy_i) \cdot (INTR_i)$

Figure 34: Interrupt Handler FSM and Condition Table

BasicTimer.vhd.3

מודול ה-BasicTimer מממש טיימר bit-32 המאפשר יצירת אירועים מחזוריים, פסיקות, אות PWM ולכידת ערך המונה בעקבות אירוע חיצוני (Capture). המודול נשלט באמצעות רגיסטרי בקרה הממופים למרחב ה-Memory-Mapped I/O, ולכן המעבד יכול להגדיר את אופן פעולת הטיימר ולקרוז את תוצאותיו באמצעות פקודות Load/Store. המודול מקבל שני קלטי Capture, CAPIN1 ו-CAPIN2, ומוציא את האותות BTIFG, Interrupt Controller ואות PWMout עבור יציאת ה-PWM.

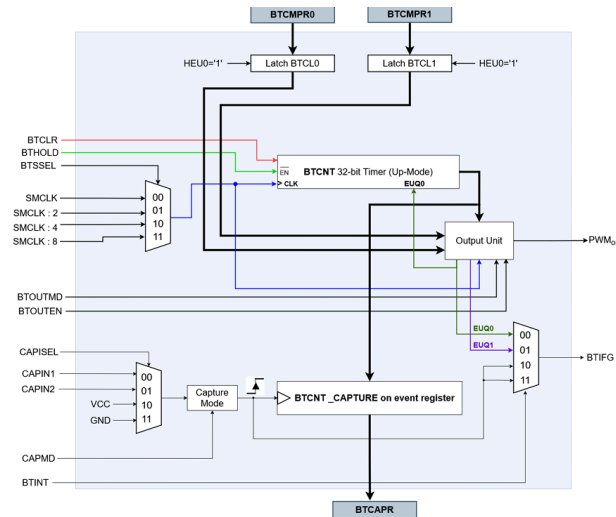
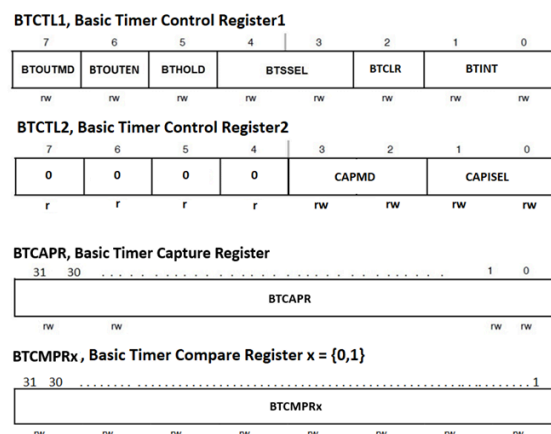


Figure 35: Basic Timer Functional Diagram

רגיסטרי ה-Basic Timer:

פעולת הטיימר נשלטת באמצעות מספר רגיסטרים. BTCTL1 ו-BTCTL2 מכילים את ביטי הבקרה, BTCMPR0 ו-BTCMPR1 מחזיקים ערכי Compare ו-BTCAPR שומר את ערך המונה שנלכד במצב Capture. בתוך המודול קיים גם המונה הפנימי BTCNT ושני רגיסטרים פנימיים BTCL0 ו-BTCL1, המשמשים כערכי ההשוואה בפועל.



7

Figure 36: Bit-level Structure of the Basic Timer Registers

BT CNT הוא מונה bit-32 המהווה את ליבת ה-Basic Timer. קצב ההתקדמות שלו נקבע על ידי השדה BTSSEL מתוך BTCTL1. המודול כולל Prescaler המבצע מניה מחזורית מ-0 עד 7 ומאפשר לבחור בין ארבעה קצבי ספירה: SMCLK, SMCLK/2, SMCLK/4, ו-SMCLK/8. עבור SMCLK מתקבל timer_tick בכל מחזור שעון, עבור SMCLK/2 כאשר ביט ה-LSB של ה-Prescaler הוא 1, עבור SMCLK/4 כאשר שני ביטי ה-LSB הם 11, ועבור SMCLK/8 כאשר ערך המונה הוא 111. בנוסף, BTHOLD מאפשר לעצור את המונה ולשמור את ערכו הנוכחי, ו-BTCLR מאפס את BT CNT ומתאפס בעצמו לאחר ביצוע פעולת האיפוס.

ה-Basic Timer כולל שני רגיסטרי Compare: BTCMPR0 ו-BTCMPR1. הערכים הנכתבים אליהם מועברים לרגיסטרים פנימיים BTCL0 ו-BTCL1, המשמשים להשוואה מול BT CNT. במימוש שלנו ערכי ה-Compare נטענים ל-BTCL0/1 כאשר המונה נמצא בערך אפס או לאחר פעולת BTCLR. כאשר BT CNT מגיע לערך ההשוואה המתאים, נוצר אירוע Compare. שני אירועים אלו, Compare0 ו-Compare1, יכולים לשמש הן ליצירת פסיקת Timer והן כחלק ממנגנון יצירת ה-PWM. כאשר מצב Capture אינו פעיל ו-BTCL0 מוגדר, BTCMPR0 משמש גם להגדרת מחזור הספירה: לאחר שהמונה מגיע לסוף המחזור הוא חוזר ל-0 ומתחיל מחזור חדש.

יצירת פסיקת: השדה BTINT בתוך BTCTL1 קובע איזה אירוע של הטיימר ייצר את אות הפסיקה BTIFG. ניתן לבחור בין אירוע Compare של BTCMPR0, אירוע Compare של BTCMPR1 או אירוע Capture. כאשר האירוע שנבחר מתרחש, המודול מייצר פולס BTIFG. פולס זה מועבר ל-Interrupt Controller, אשר שומר אותו כבקשת פסיקה ממתינה ומטפל בו בהתאם לערכי IE, GIE וסדר העדיפויות.

PWM – Output Compare Mode:

אחד מתפקידי ה-Basic Timer הוא יצירת אות PWM. במצב זה BTCMPR0 מגדיר את מחזור הספירה, בעוד BTCMPR1 קובע את נקודת שינוי רמת המוצא בתוך המחזור. כך שינוי היחס בין שני ערכי ה-Compare מאפשר לשלוט ב-Duty Cycle של אות ה-PWM. לפי דרישות הפרויקט, השדה BTOUTMD בוחר בין שני מצבי יציאה, ו-BTOUTEN קובע האם ערך PWMout מתעדכן. במימוש קיימים שני מצבים הנקבעים לפי הערכים BTCL0, BTCL1:

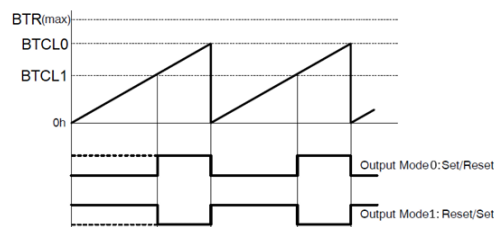


Figure 37: PWM Signal Generation using Output Compare Mode

:Capture Mode

מצב Capture מאפשר לשמור את ערך BTCNT בדיוק בזמן שמתרחש אירוע חיצוני. השדה CAPSEL מתוך BTCTL2 בוחר את מקור האירוע: CAPIN1, CAPIN2, קבוע '1' או קבוע '0'. השדה CAPMD קובע האם לזהות Falling Edge או Rising Edge או Disable עבור מצב ה-Capture. במימוש הקלט הנבחר נדגם באמצעות שני רגיסטרים עוקבים, ועל סמך הערך הנוכחי והקודם מזוהה החזית. כאשר מזוהה האירוע שנבחר, הערך הנוכחי של BTCNT נשמר בתוך BTCAPR. BTINT מוגדר לבחור ב-Capture כמקור הפסיקה, אותו אירוע גם גורם ליצירת BTIFG, ולכן ניתן להפעיל ISR בעקבות אירוע ה-Capture.

ה-Basic Timer מחובר ל-DataBUS משותף באמצעות ממשק דו-כיווני. בזמן כתיבה המעבד מעדכן את רגיסטרי הבקרה או ה-Compare בהתאם לכתובת שנבחרה. בזמן קריאה, המודל מחזיר את תוכן הרגיסטר המבוקש ורק כאשר mem read פעיל והכתובת שייכת ל-Basic Timer הוא מקבל הרשאה להניע את ה-DataBUS. בכך נמנע Bus Contention מול שאר רכיבי ה-MMIO.

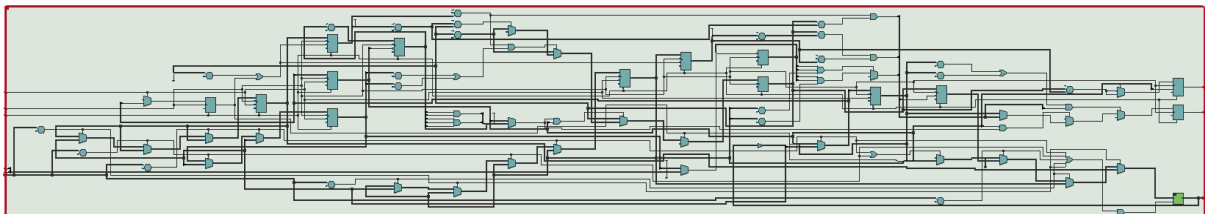


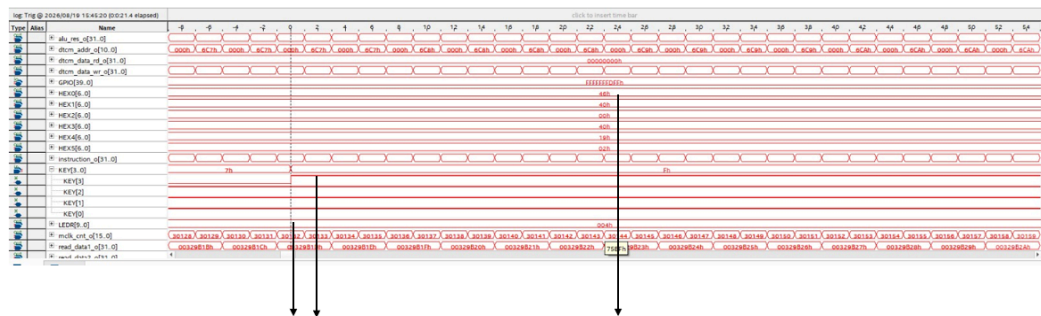
Figure 38:RTL Basic Timer Unit

ולדיציה עבור רכיבי Interrupt:

על מנת לבדוק שרכיב ה Interrupt פועלים ומגיבים בהתאם לתוכנית ביצענו בדיקות לפי ארבעת הטסטים. שקיבלנו.

טוט 1:

סטט זה בודק ביצוע פעולות מתמטיות על מערכים. בלחיצה על KEY1 מוצג איבר ממערך 1 על 0-1 HEX, בלחיצה על KEY2 מוצג איבר ממערך 2 על 2-3 HEX, ובלחיצה על KEY3 מתבצעת ריצה מחזורית המציגה את איברי המערכים ואת תוצאת החילוק (מנה ושארית) ביניהם כך שהתוצאה על 4-5 HEX והשארית מוצגת על שמונת הלדים הימניים בערכה באמצעות בקידוד בינארי.



בלחיצה על KEY3 מבוצע פעולת חילוק ומשתנה HEX0/1 והלדים

Figure 39: Signal Tap Validation for Test1

:2 606

טסט זה בודק פסיקות טיימר קבועות המקדמות מונה כל שנייה. בלחיצה על KEY1 המונה מוצג על צמד ה-HEX הראשון, בלחיצה על KEY2 הוא מוצג על הצמד השני, ובלחיצה על KEY3 על הצמד השלישי.



נשים לב כי ההקסות אכן משתנות בעת לחיצה כפתור ובנוסף יש שינוי ערכים על ידי מנייה

Figure 40: Signal Tap Validation for Test2

סטט 3:

סטט זה בודק שינוי דינמי של תדר פסיקות הטיימר. בלחיצה על KEY1 קצב הפסיקות משתנה ל-0.5 שניות, בלחיצה על KEY2 קצב הפסיקות משתנה ל-0.25 שניות, ובלחיצה על KEY3 ל-0.125 שניות (בכל לחיצה המונה גם מוצג במיקום שונה).

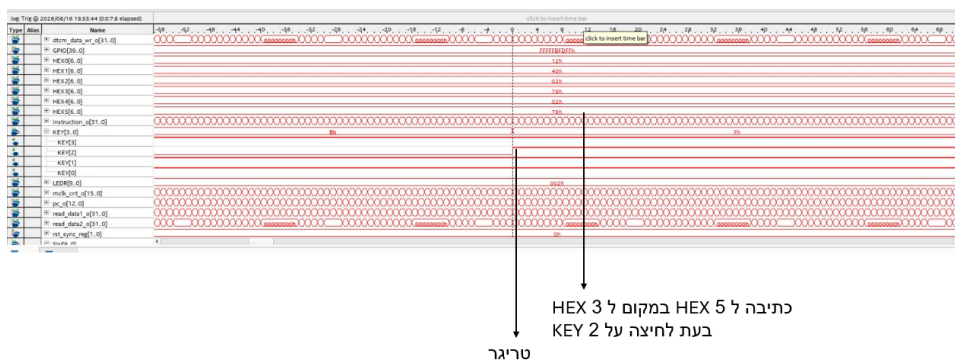


Figure 41: Signal Tap Validation for Test3

סטט 4:

סטט 4 בודק מצבי עבודה שונים ומתקדמים של הטיימר: בלחיצה על KEY1 כל לחיצה מעדכנת ברצף את מרווח הזמן בין פעימות התקדמות המונה (1 שנייה, 0.5, 0.25, ו-0.125 שניות). המונה מתקדם ב-1 כל הזמן ומוצג על גבי 0-1 HEX. דוגמה ללחיצה על KEY1 וקצב הספירה קטן פי 2

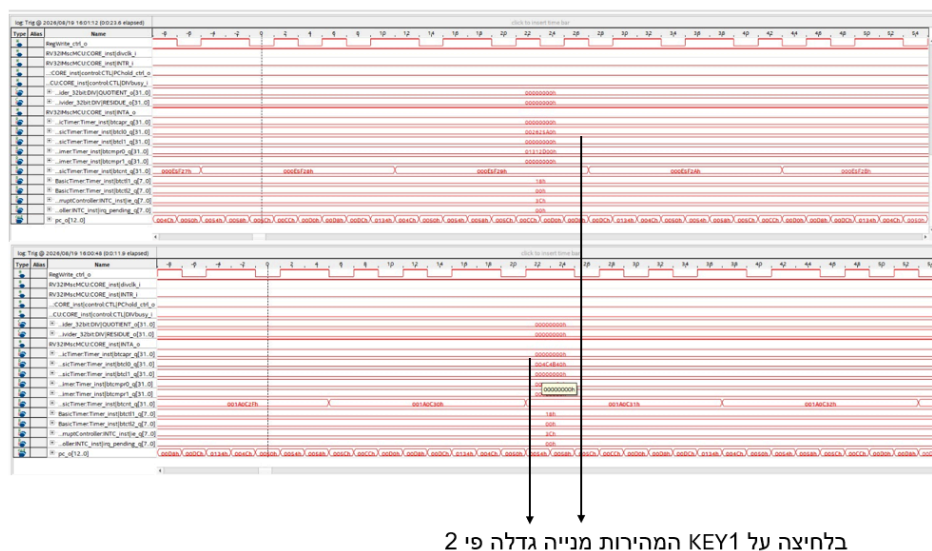
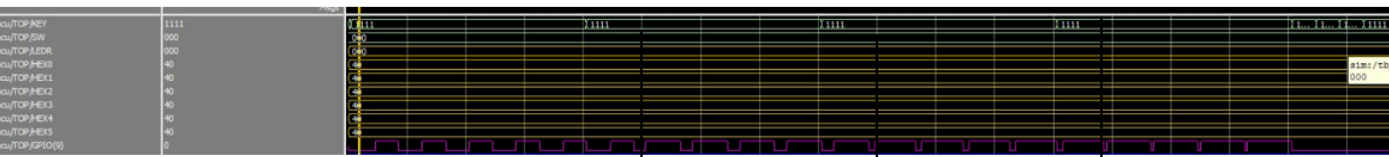


Figure 42: Signal Tap Validation for Test4 Key1

בלחיצה על KEY2 (מצב יצירת PWM): הטיימר מוגדר למצב השוואה המפיק את PWM בתדר קבוע של 5kHz. כל לחיצה משנה ברצף את מחזור העבודה (Duty Cycle) של האות בין הערכים: 75%, 50%, 87.5% ו-93.75%.

דוגמה ללחיצה על KEY 2 בתוכנת MODELSIM



לחיצה על KEY 1 . שינוי Duty Cycle

Figure 43: WaveForm Verification for Test4 Key2

כעת נראה את 4 מצבי DutyCyclen אשר בדקנו במעבדה:

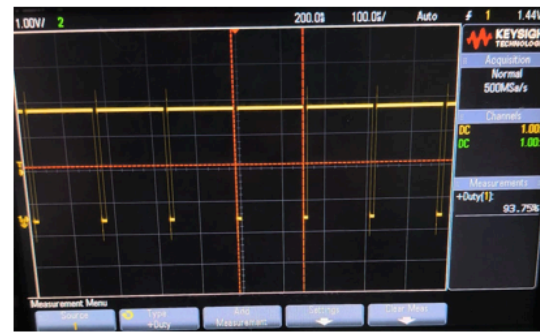
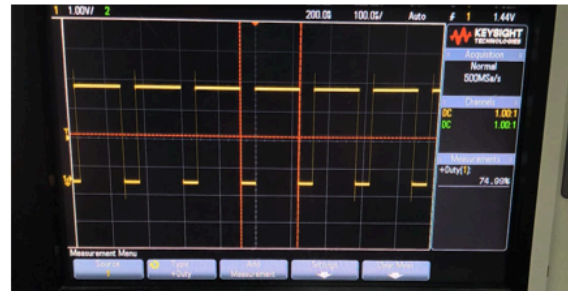


Figure 44: Duty Cycle Verification performed with Oscilloscope

בלחיצה על KEY3 הטיימר מוגדר למצב לכידה למטרת מדידת זמני ריצה של קוד. לחיצות זוגיות מודדות את הזמן שלוקח לבצע פעולת חילוק (Division) בין שני מערכים איבר-באיבר, ולחיצות אי-זוגיות מודדות את הזמן שלוקחת פעולת שארית (Modulo) בין המערכים. בנוסף כמובן אנחנו כותבים את תוצאות הפעולות למערכי הפלט בזיכרון.

Instance 1: DTCM																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
000000	00	00	30	00	00	00	32	34	00	00	32	34	00	00	32	38	00	00	31	F8	00	00	31	38	00	00	31	78	00	00	31	B8	00	00	00	03	00	00	00	51	..0...	24..	24..	28..	1...	18..	1x...	1.....	Q																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
00000a	00	00	00	52	00	00	00	53	00	00	00	54	00	00	00	55	00	00	00	56	00	00	00	57	00	00	00	58	00	00	00	59	00	00	00	5A	00	00	00	0B	...R...	S...	T...	U...	V...	W...	X...	Y...	Z...																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
000014	00	00	00	0C	00	00	00	0D	00	00	00	0E	00	00	00	0F	00	00	00	10	00	00	00	11	00	00	00	12	00	00	00	13	00	00	00	14	00	00	00	07																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
00001e	00	00	00	06	00	00	00	06	00	00	00	06	00	00	00	05	00	00	00	05	00	00	00	05	00	00	00	04	00	00	00	04	00	00	00	04	00	00	00	04																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
000028	00	00	00	0A	00	00	00	05	00	00	00	05	00	00	00	0A	00	00	00	06	00	00	00	02	00	00	00	02	00	00	00	04	00	00	00	0A	00	00	00	0A																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
000032	00	00	00	08	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

זמני הריצה

תוצאות חישוב המערכים, מנה ושארית

זמני הריצה

Figure 45: ISMCE Verification of Test4 Key3

חלק רביעי - הטמעת המערכת בכרטיס FPGA DE10

בשלב זה, לאחר תיאור המודולים המרכזיים המרכיבים את המערכת, ניתן לבחון את אופן שילובם במימוש המלא על גבי ה-FPGA. כלל רכיבי ה-MCU, ובהם ליבת ה-RV32IM, הזיכרונות, רכיבי ה-GPIO, מערכת הפסיקות, ה-Basic Timer ומאיץ המחלק, מחוברים יחד למערכת אחת באמצעות אותות בקרה, Address Bus ו-DataBUS משותף. בנוסף, המימוש כולל מערכת שעונים המפיצה שעונים נפרדים לרכיבי המערכת וחיבור ישיר למשאבי הקלט והפלט הפיזיים של לוח ה-FPGA.

קבצי VHD המרכיבים את המערכת :

FPGA_Top_SingleCycle.vhd.1

מודול ה-Top Level של המערכת ואחראי על שילוב כלל הרכיבים לכדי מערכת אחת הממומשת על גבי לוח ה-FPGA. המודול מבצע Instantiation של ליבת ה-RV32IMscMCU, רכיב ה-GPIO, ה-Interrupt Controller, ה-Basic Timer ורכיבי ה-PLL, ומחבר ביניהם באמצעות אותות הכתובת, אותות MemRead/MemWrite, אותות הפסיקה וה-DataBUS המשותף. בנוסף, המודול מהווה את שכבת החיבור בין המערכת הלוגית לבין רכיבי הלוח הפיזיים. הקלטים SW ו-KEY מחוברים לרכיבי הקלט המתאימים, בעוד LEDR ו-HEX0-HEX5 משמשים להצגת תוצאות. קווי GPIO משמשים בין היתר עבור כניסות ה-Capture של ה-Basic Timer ועבור יציאת אות ה-PWM.

נציין כי השתמשנו במנגון אשר מסנכרן את ה-Reset לשעון. בעת הפעלת הלוח המערכת נשארת ב-Reset עד ללחיצה הראשונה על KEY0, ובכך ניתן זמן להתייצבות מערכת השעונים. לאחר מכן, כל לחיצה על KEY0 יוצרת את אות rst_req_w.

מכיוון ש-KEY0 הוא קלט אסינכרוני, rst_req_w מועבר דרך שני Flip-Flops עוקבים (rst_sync_reg) המסונכרנים ל-CLOCK_50. המוצא rst_w מתקבל מהשלב השני ומשמש כ-Reset המשותף לרכיבי המערכת. כך שחרור ה-Reset מתבצע בצורה מסונכרנת ומפחית את הסיכון ל-Metastability.

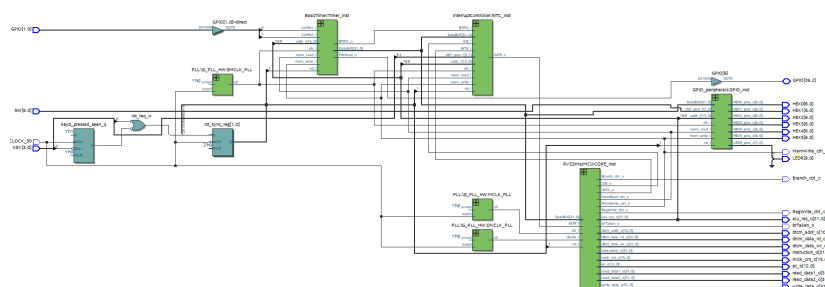


Figure 46: RTL Schematic of the Top-Level FPGA Module

PLL.vhd.2

משמש ליצירת שעוני העבודה של המערכת מתוך שעון הכניסה CLOCK_50. תדר המוצא נקבע באמצעות הפרמטרים G_PLL_MUL ו-G_PLL_DIV. את ערכי הפרמטרים קבענו כך שתדר העבודה יהיה גבוה ככל האפשר תוך עמידה בדרישות התזמון של המערכת. לצורך כך בוצע Timing Analysis ב-Quartus ובבחן ה-Worst-Case Setup Slack. כל עוד ה-Slack חיובי המערכת עומדת בדרישות התזמון, כאשר ערך חיובי הקרוב ל-0 מצביע על עבודה בסמוך לגבול התדר של המימוש.

ב-FPGA_Top_SingleCycle נעשה שימוש בשלושה מופעים של רכיב ה-PLL, היוצרים שלושה מסלולי שעון: MCLK עבור ליבת המעבד, SMCLK עבור ההתקנים ההיקפיים ו-DIVCLK עבור יחידת החילוק.

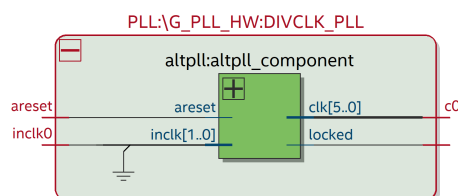


Figure 47: RTL Schematic of the System PLL Module

חלק חמישי - ניתוח ביצועים

AREA:

	Logic Elements	Registers	I/O pins	Embedded Memory bits	Embedded 9-bit Multipliers	PLL's
MCU with GPIO	1696	1691	107	131,072	3	1
MCU with GPIO and Interrupt Capability	2169	2059	107	131,072	3	1

PERFORMANCE:

	F_{max}	Critical Path	F_{mclk}
MCU with GPIO	mclk: 29.29 MHz accclk: 179.6 MHz	ITCM → Decode/Execute logic → DTCM	29.29 MHz
MCU with GPIO and Interrupt Capability	mclk: 26.96 MHz accclk: 169.72 MHz	ITCM → Decode/Control → Execute (Multiplier/ALU) → Timer → Interrupt Controller → Data Bus → DTCM	26.96 MHz

POWER:

	Total power consumption	Static power consumption	Dynamic power consumption	I/O power consumption
MCU with GPIO	442.68 mW	415.28 mW	13.77 mW	13.63 mW
MCU with GPIO and Interrupt Capability	442.81 mW	415.28 mW	13.92 mW	13.62 mW

כעת נצרף תמונות מתוכנת Quartus המוכיחות נתונים אלו:

ation Report - FPGA_Top_SingleCycle	
<<Filter>>	
Flow Status	Successful - Tue Aug 25 17:48:44 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	FPGA_Top_SingleCycle
Top-level Entity Name	FPGA_Top_SingleCycle
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	2,169 / 41,910 (5 %)
Total registers	2059
Total pins	107 / 499 (21 %)
Total virtual pins	268
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	3 / 112 (3 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	1 / 15 (7 %)
Total DLLs	0 / 4 (0 %)

AREA INTERRUPT

Figure 48: Area report MCU with Interrupt

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Aug 24 19:13:41 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	FPGA_Top_SingleCycle
Top-level Entity Name	FPGA_Top_SingleCycle
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	1,696 / 41,910 (4 %)
Total registers	1691
Total pins	107 / 499 (21 %)
Total virtual pins	268
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	3 / 112 (3 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	1 / 15 (7 %)
Total DLLs	0 / 4 (0 %)

Figure 49: Area report MCU without Interrupt

	Fmax	Restricted Fmax	
1	29.29 MHz	29.29 MHz	mclk
2	93.52 MHz	93.52 MHz	altera_reserved_tck
3	179.6 MHz	179.6 MHz	accelclk

Figure 50: Maximum frequency for MCU without Interrupt

Slow 1100mV 85C Model Fmax Summary			
<<Filter>>			
	Fmax	Restricted Fmax	Clock Name
1	26.96 MHz	26.96 MHz	\G_PLL_HW:MCLK_PLL[u0]altpll_component[a...d]generic_pll1~PLL_OUTPUT_COUNTER c
2	90.18 MHz	90.18 MHz	altera_reserved_tck
3	169.72 MHz	169.72 MHz	\G_PLL_HW:DIVCLK_PLL[u0]altpll_component...d]generic_pll1~PLL_OUTPUT_COUNTER c

Figure 51: Maximum frequency for MCU with Interrupt

POWER:

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Tue Aug 18 19:55:45 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	FPGA_Top_SingleCycle
Top-level Entity Name	FPGA_Top_SingleCycle
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	442.68 mW
Core Dynamic Thermal Power Dissipation	13.77 mW
Core Static Thermal Power Dissipation	415.28 mW
I/O Thermal Power Dissipation	13.63 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figure 52: Power Report for MCU without Interrupt

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Thu Aug 20 15:05:46 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	FPGA_Top_SingleCycle
Top-level Entity Name	FPGA_Top_SingleCycle
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	442.81 mW
Core Dynamic Thermal Power Dissipation	13.92 mW
Core Static Thermal Power Dissipation	415.28 mW
I/O Thermal Power Dissipation	13.62 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figure 53: Power Report for MCU with Interrupt

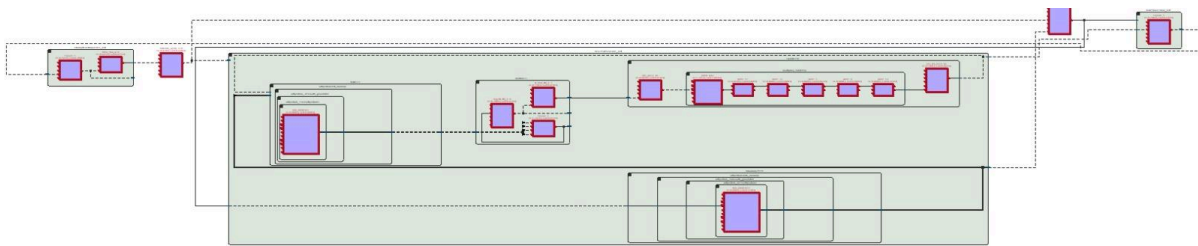


Figure 54: Critical path Report for MCU with Interrupt

Command Info	Summary of Paths						
Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	
1 1.451	RV32IMcMCU.CORE_inst[fetch:IFE]altsyncr_inst_memory[altsyncr...generated[altsyncr_7563altsyncr]ram_block3a5-ports_we_reg	RV32IMcMCU.CORE_inst[dmemory.MEM_1]ram_block3a15-ports_datain_reg4	VG_PLL_1divclk	VG_PLL_1divclk	20.000	-1.460	

Path #1: Setup slack is 1.451					Path #1: Setup slack is 1.451				
Path Summary	Statistics	Data Path	Waveform	Extra Filter Information	Path Summary	Statistics	Data Path	Waveform	Extra Filter Information
1	From Node	RV32IMcMCU.CORE_inst[fetch:IFE]altsyncr_563altsyncr]ram_block3a5-ports_we_reg			1	From Node	RV32IMcMCU.CORE_inst[fetch:IFE]		
2	To Node	RV32IMcMCU.CORE_inst[dmemory.MEM]altsyncr_1]ram_block3a15-ports_datain_reg4			2	To Node	RV32IMcMCU.CORE_inst[dmemory]		
3	Launch Clock	VG_PLL_HWMCLK_PLL[u0]altpll_component[a_d]generic_pll1-PLL_OUTPUT_COUNTER]divclk			3	Launch Clock	VG_PLL_HWMCLK_PLL[u0]altpll_com		
4	Latch Clock	VG_PLL_HWMCLK_PLL[u0]altpll_component[a_d]generic_pll1-PLL_OUTPUT_COUNTER]divclk (INVERTED)			4	Latch Clock	VG_PLL_HWMCLK_PLL[u0]altpll_com		
5	Data Arrival Time	21.788			5	Data Arrival Time	21.788		
6	Data Required Time	23.239			6	Data Required Time	23.239		
7	Slack	1.451			7	Slack	1.451		

Figure 55: Critical path Report for MCU with Interrupt

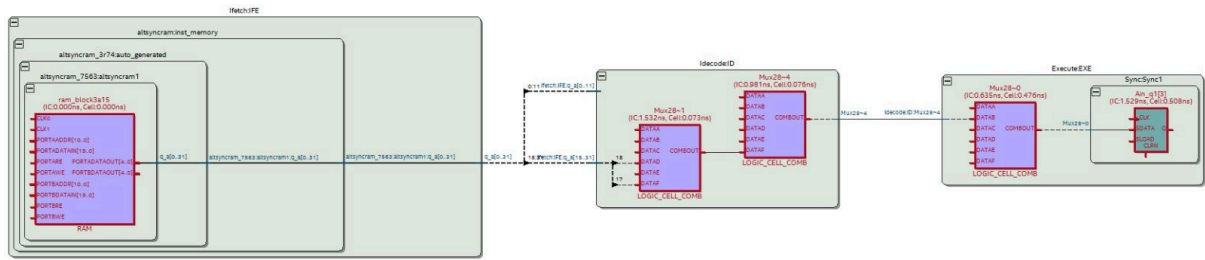


Figure 56: Critical path Report for MCU without Interrupt

1	2.017	RV32IMscMCU.CORE_inst[fetchIFE]altsyncram_inst_memoryaltsyncr_generatedaltsyncram_7563altsyncram1[ram_block3a15-portsa_we_reg]	RV32IMscMCU.CORE_inst[ExecuteEXE]SyncSync1[Ain_q[3]]	Launch Clock	Latch Clock	Relationship	Slack
				mcclk	acclk	11.000	-2.183

Path #1: Setup slack is 2.017		Path #1: Setup slack is 2.017	
Property	Value	Property	Value
1 From Node	RV32IMscMCU.CORE_inst[fetchIFE]altsyncr_63altsyncram1[ram_block3a15-portsa_we_reg]	1 From Node	RV32IMscMCU.CORE_inst[fetchIFE]
2 To Node	RV32IMscMCU.CORE_inst[ExecuteEXE]SyncSync1[Ain_q[3]]	2 To Node	RV32IMscMCU.CORE_inst[ExecuteEXE]
3 Launch Clock	mcclk	3 Launch Clock	mcclk
4 Latch Clock	acclk	4 Latch Clock	acclk
5 Data Arrival Time	11.557	5 Data Arrival Time	11.557
6 Data Required Time	13.574	6 Data Required Time	13.574
7 Slack	2.017	7 Slack	2.017

Figure 57: Critical path Report for MCU without Interrupt

סיכום ביצועים וניתוח השוואתי (Performance & Timing Analysis)
הוספת מנגנון הפסיקות (Interrupt Capability) והטיימר למערכת מציגה Trade-off הנדסי בין הרחבת הפונקציונליות לבין ביצועי התזמון, ניצול המשאבים וצריכת ההספק.
שילוב התמיכה בפסיקות הוביל לירידה של כ-8% בתדר השעון המקסימלי של ה-MCLK, מ-29.29 MHz ל-26.96 MHz. ניתוח ה-Critical Path ב-TimeQuest מראה כי בגרסה התומכת בפסיקות הניתב הקריטי עובר דרך שרשרת קומבינטורית ארוכה הכוללת ITCM → Decode/Control → Execute.
הרכיבים המרכזיים בניתוח הם: ALU/Multiplier → Timer → Interrupt Controller → Data Bus → DTICM וה-routing לאורך מסלול זה מגדיל את ה-Propagation Delay ולכן מגביל את תדר העבודה המקסימלי. בנוסף, התמיכה בפסיקות ובטיימר הגדילה את ניצול המשאבים ב-473 Logic Elements וב-368 Registers, כתוצאה מהוספת בקר הפסיקות, הטיימר ולוגיקת הבקרה וה-Data Bus הנלווית. בהתאם, נרשמה עלייה קלה בהספק הדינמי מ-13.77 mW ל-13.92 mW, בעוד שההספק הסטטי נותר ללא שינוי בדוח Quartus.